REFERÊNCIA · IMPRIMIR E SALVAR COMO PDF

Guia de montagem

A ordem para criar de novo o que já existe na Bancada, no Sumiu e no Todo. Primeiro a API. Depois o front. Cada passo termina numa condição observável.

Os três projetos são o mesmo módulo, com nomes diferentes. Bancada chama o recurso de Ordem. Sumiu chama de Requisition. Todo chama de Tarefa. A pasta muda de `API/` para `backend/`. A interface HTTP é a mesma forma: um `Router` pendurado no `index.js` com `app.use`.

> Para virar PDF: abra este arquivo no navegador, Imprimir, destino "Salvar como PDF". O quiz das lições não entra na impressão.

Vocabulário

Um **módulo** tem uma **interface** (o que quem chama precisa saber) e uma **implementação** (o que fica dentro). O **seam** é o lugar onde essa interface mora: dá para trocar o que está atrás sem editar quem chama. Um **adapter** é a coisa concreta sentada nesse seam.

No Sumiu, a interface do usuário é o conjunto de chamadas em `routes/user.routes.js`. O controller é a implementação. `db/conn.js` é o adapter do Mongo. No front, `src/api.js` é o adapter dessa mesma interface: as páginas chamam `createUser` e não sabem a URL.

Profundidade é quanta capacidade cabe atrás de uma interface pequena. `axios.create` num arquivo só, com `baseURL` e `withCredentials`, é profundo. Cada página montando o próprio cliente seria raso: a interface vazou para todo mundo.

1. Criar a API

1.1 Pasta e manifesto

```
mkdir api && cd api
npm init -y
```

`npm init -y` escreve o `package.json` com os padrões, sem perguntas. Fonte: npm-init. No arquivo gerado, deixe `"type": "module"` para os `import` que os três projetos usam, e dois scripts:

```
"type": "module",
"scripts": {
  "dev": "nodemon index.js",
  "start": "node index.js"
}
```

> Pronto quando `node -e "console.log(1)"` roda na pasta e o `package.json` tem `"type": "module"`.

1.2 Pacotes do núcleo

```
npm install express mongoose dotenv cors cookie-parser
npm install -D nodemon
```

PACOTE	O QUE FAZ NESTA API	ONDE APARECE
`express`	Cria o servidor HTTP e entrega `req` / `res` para cada rota. Instalação.	Os três
`mongoose`	Abre o Mongo e define o model. Conexão, models.	Os três
`dotenv`	Lê o `.env` para `process.env`. A primeira linha do `index.js` é `import "dotenv/config"`. dotenv.	Sumiu, Todo. A Bancada lê `process.env` e não importa o dotenv no `index.js`.
`cors`	Autoriza o navegador em `localhost:5173` a chamar a API, com cookie. `credentials: true` e `origin` juntos. cors.	Os três
`cookie-parser`	Preenche `req.cookies`. Sem ele o middleware de login não vê o token. cookie-parser.	Os três instalam. Sumiu e Todo usam.
`nodemon`	Reinicia o processo quando um arquivo muda. Fica em `devDependencies`. O script `dev` chama ele.	Os três

> Pronto quando `npm ls express mongoose dotenv cors cookie-parser` lista os cinco, e `nodemon` aparece só em dev.

1.3 Variáveis

Crie `.env` na pasta da API. Não commite. O Todo guarda o modelo em `backend/.env.example`.

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/nome_do_banco
FRONTEND_URL=http://localhost:5173
```

Sumiu e Todo caem em 5000 se `PORT` faltar. A Bancada cai em 3000. O front do Vite sobe em 5173, e esse tem de ser o `origin` do CORS.

> Pronto quando o arquivo existe e o `.gitignore` contém `.env`.

2. Arquivos da API, nesta ordem

A ordem é a ordem das dependências. Um model que importa a conexão não existe antes dela. Uma rota que aponta para um controller não existe antes dele. O `index.js` só monta o que já foi exportado. Apagar o `index.js` e deixar models e rotas espalhados não sobe servidor nenhum: a complexidade volta para quem teria de chamar cada arquivo. O `index.js` é o único lugar que compõe.

2.1 `db/conn.js` — adapter do banco

Os três arquivos são o mesmo: `mongoose.connect(process.env.MONGO_URI)` e `export default mongoose`. O model importa esse módulo, não a string de conexão.

```
import mongoose from "mongoose";

const main = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("Conectado ao MongoDB");
};

main().catch((err) => console.error("Erro ao conectar ao MongoDB", err));

export default mongoose;
```

> Pronto quando, com o Mongo no ar, o processo imprime “Conectado ao MongoDB”. Ainda não há rota.

2.2 `models/` — um schema por substantivo

Um arquivo, um `mongoose.model`. Campos `required` são a interface do documento: quem chama `save` precisa deles. No Sumiu o usuário tem `name`, `email`, `password` com `select: false`. A ordem tem `user` como `ObjectId` apontando para `"User"`. A tarefa do Todo aponta para o usuário da mesma forma.

Importe o mongoose de `../db/conn.js`, como a Bancada e o model de usuário do Sumiu. Importar direto de `"mongoose"` também funciona depois da conexão, e o model de requisição do Sumiu faz isso. Um caminho só deixa o seam óbvio.

> Pronto quando `mongoose.model("User", schema)` exporta o model e o processo ainda conecta, sem rota nenhuma.

2.3 `controllers/` — a implementação

Uma classe com métodos estáticos, ou métodos de instância, que recebem `(req, res)`. O controller importa o model e devolve JSON. Ele não registra URL. A Bancada faz o CRUD inteiro aqui (`create`, `find`, `findById`, `findByIdAndUpdate`, `findByIdAndDelete`). O controller da Bancada usa `User` e precisa importar o model: sem esse import o arquivo não executa.

> Pronto quando cada método lê `req.body` ou `req.params` e responde com `res.status(...).json(...)`, e o model está importado no topo.

2.4 `routes/` — a interface HTTP

`Router()` amarra método e caminho a um método do controller. Isso é o que o front precisa saber. Fonte: Routing.

```
import { Router } from "express";
import UserController from "../controllers/user.controller.js";

const userRoutes = Router();

userRoutes.post("/", UserController.postUser);
userRoutes.post("/login", UserController.login);

export default userRoutes;
```

Caminho fixo antes de caminho com parâmetro. Na Bancada, `GET /ordens/status/:status` está declarado depois de `GET /ordens/:id`. `:id` engole o segmento `status`. Declare `/status/:status` primeiro.

> Pronto quando o arquivo exporta o router e cada linha aponta para um método que existe no controller.

2.5 `index.js` — compõe e escuta

A ordem dentro do arquivo também é obrigatória. `express.json()` antes das rotas, senão `req.body` chega vazio. `cookieParser` antes de qualquer middleware que leia cookie. `cors` antes das rotas. Fonte: Using middleware.

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";
import userRoutes from "./routes/user.routes.js";

const app = express();

app.use(express.json());
app.use(cookieParser());
app.use(cors({
  credentials: true,
  origin: process.env.FRONTEND_URL || "http://localhost:5173",
}));

app.use("/users", userRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Um arquivo de rota que ninguém importa não responde. O `app.use("/users", userRoutes)` é o que publica a interface. Some o prefixo com o caminho do router: `post("/")` vira `POST /users`, `post("/login")` vira `POST /users/login`.

O Todo e o Sumiu embrulham o `app` em `createServer` porque o Socket.IO precisa do servidor HTTP. Sem socket, `app.listen` basta. A Bancada faz assim.

> Pronto quando `npm run dev` imprime a porta e um `POST /users` com JSON válido devolve 200 ou 201.

3. O módulo de usuário, com sessão

A Bancada grava a senha como veio e não tem middleware. Sumiu e Todo são o modelo a repetir. Instale quando for escrever o login, não antes: o CRUD do passo 2 já sobe sem isso.

```
npm install argon2 jsonwebtoken
```

PACOTE	O QUE FAZ
`argon2`	`argon2.hash` no cadastro, `argon2.verify` no login. A senha que vai para o Mongo já é hash. node-argon2.
`jsonwebtoken`	`jwt.sign` com `JWT_SECRET` depois que a senha confere. `jwt.verify` no middleware. jsonwebtoken.

No `.env`, some `JWT_SECRET` e `JWT_EXPIRATION_MS`. O login do Sumiu grava o token num cookie `httpOnly` chamado `token`. O middleware em `middleware/user.js` lê `req.cookies.token`, verifica, põe `req.user` e chama `next()`. Sem cookie, responde 401.

O seam fica na rota, não espalhado nos controllers:

```
userRoutes.post("/logout", userMiddleware, UserController.logout);
userRoutes.put("/:id", userMiddleware, UserController.updateUser);
```

No Todo, criar e listar tarefa passam pelo mesmo middleware. No Sumiu, as rotas de requisição ainda não passam. Quando a tela exigir usuário logado, o lugar de declarar isso é a linha da rota.

No login, o `findOne` precisa de `.select("+password")` porque o schema do Sumiu marca `password` com `select: false`.

> Pronto quando um `POST /users/login` devolve `Set-Cookie`, e um `POST /users/logout` sem esse cookie devolve 401.

4. O recurso

Repita 2.2–2.4 com o segundo substantivo. No Sumiu: `models/requisition.js`, controller, `routes/requisition.routes.js`, e no `index.js` a linha `app.use("/requisitions", requisitionRoutes)`. Na Bancada o prefixo é `/ordens`. No Todo é `/tarefas`, e cada linha da rota leva `userMiddleware`.

A interface mínima, a que a Bancada e o Sumiu expõem:

CHAMADA	EFEITO
`POST /`	cria
`GET /`	lista
`GET /:id`	um documento
`PUT /:id`	substitui campos
`DELETE /:id`	apaga

O Todo troca o `PUT` por `PATCH /:id/situacao`. É a mesma ideia: uma linha no router, um método no controller.

> Pronto quando `GET` no prefixo devolve um array JSON, mesmo que vazio.

5. Criar o front

Só comece com a API respondendo `POST` de usuário e `GET` da lista. A tela é um adapter da interface que você já publicou.

5.1 Vite

```
cd ..
npm create vite@latest front -- --template react
cd front
npm install
```

Fonte: guia do Vite. Isso gera `index.html` com `<div id="root">` e `src/main.jsx`, que chama `createRoot`. `npm run dev` sobe em `http://localhost:5173`.

> Pronto quando o navegador abre a página gerada pelo Vite.

5.2 Pacotes da interface

```
npm install react-router-dom axios
npm install tailwindcss @tailwindcss/vite
```

PACOTE	O QUE FAZ
`react` e `react-dom`	Já vêm do template. A função devolve JSX. Describing the UI.
`react-router-dom`	`BrowserRouter` no `main.jsx`, `Routes` e `Route` no `App.jsx`. Uma URL, uma página. Routing.
`axios`	O cliente HTTP. `withCredentials: true` manda o cookie que o login gravou. Axios.
`tailwindcss` e `@tailwindcss/vite`	Classes no JSX. No `vite.config.js`, `plugins: [react(), tailwindcss()]`. No `src/index.css`, a primeira linha é `@import "tailwindcss";`. Instalação com Vite.

`react-hot-toast` está no Sumiu e na Bancada. É aviso visual, não faz parte da interface. Entra quando uma tela precisar de mensagem, com `npm install react-hot-toast`.

> Pronto quando `npm run dev` continua no ar depois dos dois `npm install`.

5.3 `.env` do front

```
VITE_API_URL=http://localhost:5000
```

O Vite só expõe variável que começa com `VITE_`. O Sumiu lê `import.meta.env.VITE_API_URL`. O Todo leu o mesmo valor com o nome `VITE_BASE_URL`. Escolha um nome e use esse no `axios.create`. Reinicie o `npm run dev` depois de criar o arquivo.

> Pronto quando o valor no código é a mesma origem da API, porta inclusive.

5.4 `src/api.js` — adapter

Um `axios.create` . Funções nomeadas, uma por chamada da API. As páginas importam a função, não o axios.

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
  headers: { "Content-Type": "application/json" },
});

export const createUser = (user) => api.post("/users", user);
export const loginUser = (user) => api.post("/users/login", user);
export const getAll = () => api.get("/requisitions");

export default api;
```

Troque `/users` e `/requisitions` pelo prefixo do projeto: `/usuarios` e `/tarefas` no Todo, `/ordens` na Bancada.

> Pronto quando cada função exportada corresponde a uma linha que existe no router da API.

5.5 Router, depois as páginas

Em `src/main.jsx`, envolva `<App />` com `<BrowserRouter>`. Os três fronts fazem isso. Em `App.jsx`, declare as rotas na ordem em que você as constrói:

1. `/user/register` chama `createUser`. Corpo: nome, e-mail, senha.
2. `/user/login` chama `loginUser`. O cookie volta sozinho porque `withCredentials` está no cliente e `credentials: true` está no CORS.
3. A lista chama o `GET` da coleção e desenha o array.
4. Criar, ver um, editar: uma página por chamada, cada uma importando uma função de `api.js`.

O Sumiu já tem esse mapa em `FrontEnd/front/src/App.jsx`: `/`, `/user/login`, `/user/register`, `/requisition`, `/requisition/create`, `/requisition/edit/:id`, `/requisition/:id`. A rota com `:id` fica por último dentro do grupo, pelo mesmo motivo do `GET /:id` na API.

Uma página é uma função. Estado dos campos com `useState`. No submit, `preventDefault`, chama a função de `api.js`, e `useNavigate` manda para a próxima URL. Fonte: Describing the UI.

> Pronto quando o cadastro cria um documento, o login grava o cookie, e a lista mostra o que o `GET` devolveu.

6. Fora desta sequência

Entram depois que usuário e um recurso respondem.

PACOTE	PROJETO	PARA QUÊ
`swagger-ui-express` e `swagger-autogen`	Os três	`GET /docs`. Gere o JSON com o script `swagger` depois que as rotas existirem.
`nodemailer`	Todo	E-mail de redefinição de senha. Variáveis `EMAIL_*`.
`socket.io` e `socket.io-client`	Todo	Chat. O `index.js` passa a escutar com `httpServer`, não com `app.listen`.
`apexcharts` e `react-apexcharts`	Todo	Gráfico de situação. Só lê dados que a lista já devolve.

Mapa rápido

ORDEM	ARQUIVO	PAPEL
1	`package.json` + `.env`	manifesto e segredos
2	`db/conn.js`	adapter do Mongo
3	`models/`	formato do documento
4	`controllers/`	implementação
5	`routes/`	interface HTTP
6	`middleware/user.js`	seam do cookie
7	`index.js`	compõe e escuta
8	Vite + `src/api.js`	adapter do front
9	`main.jsx`, `App.jsx`, páginas	uma tela por chamada